

My AI-Built App Broke After an Update

— What To Do in the Next 30 Minutes

For apps built with Lovable, Bolt, Cursor, Replit, v0, Claude Code, or similar AI coding tools. If your app suddenly broke after an AI edit, update, deploy, or new feature prompt, do not keep prompting randomly. That usually makes the problem worse. Use this 30-minute recovery checklist before you ask the AI to fix anything else.

1. Stop changing the app

Do not:

- ☐ Add another feature
- ☐ Rewrite the whole app
- ☐ Spam “fix this” prompts
- ☐ Change database settings blindly
- ☐ Delete tables, auth rules, environment variables, or routes
- ☐ Ask the AI to refactor unrelated code
- ☐ Push more changes to production before understanding the break

Your first job is not to fix. Your first job is to understand what broke.

2. Identify the break type

Choose the closest match:

- ☐ UI/layout broke
- ☐ Login/signup broke
- ☐ Database/data is not loading
- ☐ App works in preview but not production
- ☐ Payment/checkout broke
- ☐ One page or route broke
- ☐ Whole app crashed
- ☐ New feature broke an old feature
- ☐ AI keeps fixing one thing and breaking another
- ☐ Error message appeared after deploy
- ☐ Supabase/Firebase connection issue
- ☐ Environment variable/config issue

☐ I do not know what broke

3. Capture the evidence

Before asking the AI to fix anything, write down:

App/tool used: Example: Lovable, Bolt, Cursor, Replit, v0, Claude Code

What was working before:

What is broken now:

When was the app last working?

What was the last prompt/change/update/deploy before it broke?

Exact error message, if any:

Does the issue happen in:

- ☐ Preview only
- ☐ Production only
- ☐ Both preview and production
- ☐ Local environment only
- ☐ Not sure

Affected page/feature:

Expected behavior:

Actual behavior:

Screenshots/logs collected:

- ☐ Screenshot
- ☐ Console error
- ☐ Network error
- ☐ Build/deploy log
- ☐ Database/auth error
- ☐ No logs yet

4. Find the last working version

Ask yourself:

- ☐ When was the app last working?
- ☐ What was the last prompt you gave?
- ☐ What files/components changed?
- ☐ Did you change Supabase/Firebase?
- ☐ Did you change environment variables?
- ☐ Did you deploy after the change?
- ☐ Did the AI touch unrelated files?

- ☐ Did the issue start immediately after one prompt?
- ☐ Did multiple things break at once?

If you cannot answer these, do not ask for a fix yet. Ask for diagnosis first.

5. Decide: rollback or fix forward

Rollback if:

- ☐ The app was working before one recent change
- ☐ The issue started immediately after one prompt
- ☐ You do not understand what changed
- ☐ Multiple pages broke at once
- ☐ The AI changed too many files
- ☐ The app is live and users are affected
- ☐ Auth, payments, or user data are involved

Fix forward if:

- ☐ The issue is small and isolated
- ☐ You know the exact broken component/page
- ☐ Only one validation, state, route, or API call is broken
- ☐ The rest of the app is stable
- ☐ You understand what changed
- ☐ You can test the fix safely

Simple rule: If one recent change broke many things, rollback first. If one small thing broke and you understand it, fix forward.

6. Use this diagnosis prompt first

Paste this into Cursor, Claude, Lovable, Bolt, or your AI coding tool:

•

You are a senior debugging engineer.

My AI-built app broke after a recent change.

Do not rewrite the whole app.

Do not refactor unrelated code.

Do not change database schema, auth rules, environment variables, routing, or payment logic unless clearly required.

First diagnose the likely root cause.

Context:

Tool used:

[Lovable / Bolt / Cursor / Replit / v0 / Claude Code / Other]

App type:

[What the app does]

What was working before:

[Describe]

What broke:

[Describe]

When it last worked:

[Describe]

Last prompt/change/update/deploy:

[Describe]

Error message:

[Paste exact error]

Affected page/feature:

[Describe]

Expected behavior:

[Describe]

Actual behavior:

[Describe]

Does it break in preview, production, or both?

[Describe]

Give me:

1. likely root cause
2. confidence level: high, medium, or low
3. what not to touch
4. rollback vs fix-forward recommendation
5. safest first fix
6. tests to confirm it is fixed
7. red flags that require a senior developer

7. Use this safe-fix prompt only after diagnosis

Do not use this before diagnosis. Paste this only after the AI explains what likely broke:

•

Fix only the smallest possible part of the issue. Do not refactor unrelated code. Do not rewrite the app. Do not change database schema, auth rules, environment variables, routing, or payment logic unless required. Before making changes, explain: 1. what file/component you will change 2. why that is the smallest safe fix 3. what you will not touch 4. how I can test the fix 5. what could break if this fix is wrong

8. Test after every fix

Check:

- ☐ Main user flow works
- ☐ Signup/login works
- ☐ Dashboard/data loading works
- ☐ Mobile layout works
- ☐ Old feature that previously worked still works

- ☐ Preview and production behave the same
- ☐ Error/loading states are acceptable
- ☐ Console has no new major errors

- ☐ Payment/checkout still works, if relevant
- ☐ No private user data is exposed
- ☐ The fix did not create a new unrelated bug

Do not continue building until the broken flow is stable again.

9. When to stop and get help

Stop and get help if:

- ☐ Auth is exposing private data
- ☐ Database permissions are confusing
- ☐ Payments or user data are involved
- ☐ App works in preview but not production
- ☐ Every fix creates a new break
- ☐ You do not understand what the AI changed
- ☐ The AI wants to rewrite large parts of the app
- ☐ There is possible data loss
- ☐ You are changing production settings blindly

10. Need a second pair of eyes?

Get VibeFix Broken App Diagnosis.

I'll review what broke, identify the likely root cause, tell you whether to rollback or fix forward, and give you exact AI prompts to repair the issue safely.

VibeFix Broken App Diagnosis — \$79 Beta

For the first 5 reports only.

You get:

- likely root cause
- confidence level
- rollback vs fix-forward recommendation
- what not to touch
- safest first fix

- priority fix order
- exact AI repair prompts
- post-fix testing checklist
- escalation warning if the issue needs a senior developer

Get My Diagnosis:

Coming soon

Important:

VibeFix is not full production support, penetration testing, legal/compliance review, 24/7 emergency support, or guaranteed bug fixing.

It is a structured diagnosis for common break patterns in AI-built apps.